/new

Scaffold a new project: /new –language –type [–stack] [–github | –gitlab] | /new help

Overview

`/new` scaffolds a new project in the current directory, which must be empty. The scaffold consists of:

- The ragent workspace: `.ragent/`, `specs/`, `log/`, `.gitignore`, `AGENTS.md`
- A runnable hello-world artifact set for 26 application languages (`rust`, `python`, `go`, `typescript`, `shell`, …) with `library`, `cmdline`, `tui`, and `gui` layouts
- Sample-document stubs for 20 data and DSL formats (`json`, `yaml`, `sql`, `cmake`, `maven`, …) covering every codeindex scanner language
- Optional stack layers via `--stack <name>`
- Starter documentation: `README.md`, `QUICKSTART.md`, `STATS.md`, `docs/`
- Git initialisation with an initial commit
- Optional GitHub or GitLab repository creation and push via `--github` / `--gitlab`

The command runs on a foreground worker thread: no sub-agent is spawned and no LLM turn is consumed. Progress streams live in the message window.

Syntax

```
/new --language <lang> --type <type>
/new --language <lang> --type <type> --stack <name>
/new --language <lang> --type <type> --github
/new --language <lang> --type <type> --gitlab
/new help
```

Options

Option	Description
`--language <lang>`	Language id. 46 canonical ids are accepted, including aliases such as `sh` -> `shell` and `yml` -> `yaml`
`--type <type>`	Layout: `library`, `cmdline`, `tui`, `gui`, or `webapp`. Data and DSL languages degrade to manifest-only scaffolds
`--stack <name>`	Optional stack layer. Rust stacks: `axum`, `warp`, `raylib`, `gtk4`, `ratatui`
`--github`	Create a private GitHub repository, set it as `origin`, and push
`--gitlab`	The same flow for GitLab. `--github` and `--gitlab` are mutually exclusive
`help`	Print the on-screen option summary

Language and layout notes

- Application languages produce a runnable hello-world artifact set.
- Data and DSL languages produce sample-document stubs instead of runnable code, and their scaffolds degrade to manifest-only.
- The stub set covers every codeindex scanner language.

Examples

Scaffold a Rust command-line project:

```
/new --language rust --type cmdline
```

Scaffold a Rust TUI project with the ratatui stack:

```
/new --language rust --type tui --stack ratatui
```

Scaffold a Python library:

```
/new --language python --type library
```

Scaffold a Rust command-line project and host it on GitHub:

```
/new --language rust --type cmdline --github
```

Scaffold a Go command-line project and host it on GitLab:

```
/new --language go --type cmdline --gitlab
```

Run `/new help` for the on-screen option summary.

Output

The worker thread streams progress lines and updates them in place each frame:

- `[ .. ] <step>` - step in progress
- `[ ok ] <step>` - step completed
- `[fail] <step>` - step failed

The final summary prints as `From: /new`.

Guards

- The current directory must be empty before scaffolding starts.
- The emitter never silently overwrites existing files.

Related

- Howto manual: `docs/howtos/newproj.md`
- `/spec` - spec lifecycle commands; `/spec reverse --create` chains into `/spec create`
- `/spec reverse` - reverse-engineer a repo and seed a spec